

CS-338 Intro to the Theory of Computation

- Lecture #6

➤ **Context Free Grammars and Pushdown Automata Equivalence**

- Prof Pietro S. Oliveto

Department of Computer Science and Engineering

Southern University of Science and Technology (SUSTech)

- `olivetop@sustech.edu.cn`
<https://faculty.sustech.edu.cn/olivetop>
- Office: Room 113

18.404/6.840 Lecture 6

Last time:

- Context free grammars (CFGs)
- Context free languages (CFLs)
- Pushdown automata (PDA)

Today: (Sipser §2.2)

- CFGs and PDAs equivalence

Context Free Grammars

$$G_1 \quad \left. \begin{array}{l} S \rightarrow 0S1 \\ S \rightarrow R \\ R \rightarrow \varepsilon \end{array} \right\} \text{(Substitution) Rules}$$

Shorthand:

$$S \rightarrow 0S1 \mid R$$

$$R \rightarrow \varepsilon$$

In G_1 :

3 rules

R, S

$0, 1$

S

Rule: Variable $\rightarrow$ string of variables and terminals

Variables: Symbols appearing on left-hand side of rule

Terminals: Symbols appearing only on right-hand side

Start Variable: Top left symbol

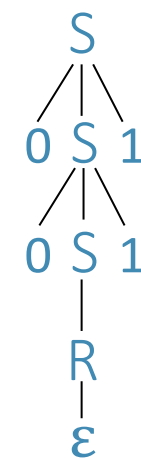
Grammars generate strings

1. Write down start variable
2. Replace any variable according to a rule
Repeat until only terminals remain
3. Result is the generated string
4. $L(G)$ is the language of all generated strings
5. We call $L(G)$ a Context Free Language.

All languages that are generated by a CGG are **context free**

Example of G_1 generating a string

Parse Tree of substitutions



S

Resulting string

$0S1$

$00S11$

$00R11$

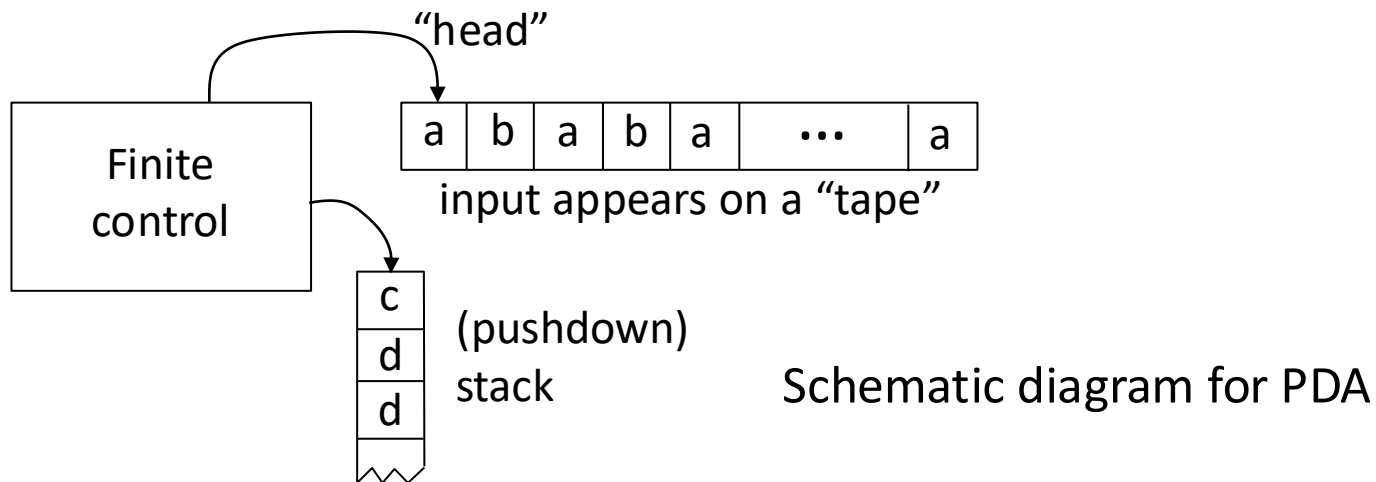
$0011 \in L(G_1)$

Derivation:

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 00R11 \Rightarrow 0011$$

$$L(G_1) = \{0^k 1^k \mid k \geq 0\}$$

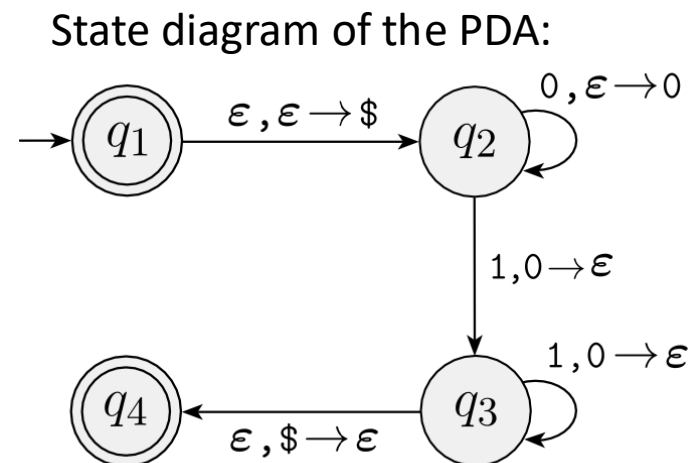
Pushdown Automata (PDA)



Example: PDA for $D = \{0^k 1^k \mid k \geq 0\}$

The unlimited nature of the stack allows the PDA to store numbers of unbounded size

- 1) Read 0s from input, push onto stack until read 1.
- 2) Read 1s from input, while popping 0s from stack.
- 3) Enter accept state if stack is empty. (note: acceptance only at end of input)



PDA – How powerful?

Which is more powerful?

- DFA or PDA?
- Regular expressions or PDA?
- NFA or PDA?
- Regular expressions or CFGs?
- **CFGs or PDAs?**

Next: Show that CFGs and PDAs are equivalent

- Both are capable of describing Context Free Languages
- We show how to convert any CFG in a PDA that recognises the same language and viceversa
- Recall: CFL are any language that can be described by a CFG.

Equivalence of CFGs and PDAs

Theorem: A is a CFL iff* some PDA recognizes A



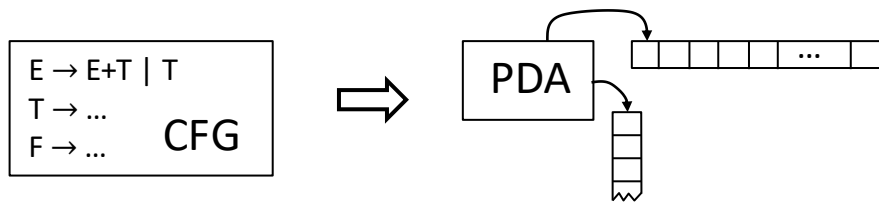
* “iff” = “if and only if” means the implication goes both ways.

So we need to prove both directions: forward ($\rightarrow$) and reverse ($\leftarrow$).

Converting CFGs to PDAs (1)

Theorem: If A is a CFL then some PDA recognizes A

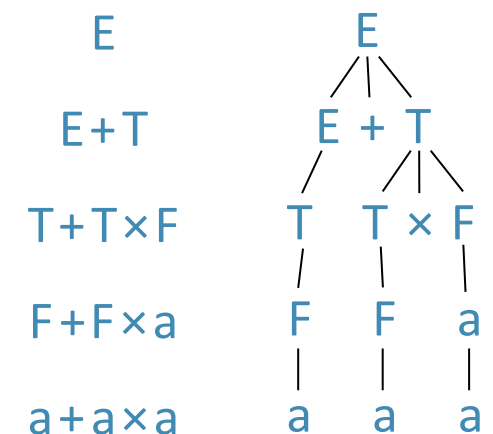
Proof: Convert A 's CFG to a PDA



Input:

a	+	a	x	a
---	---	---	---	---

G_2 $E \rightarrow E+T \mid T$
 $T \rightarrow T \times F \mid F$
 $F \rightarrow (E) \mid a$



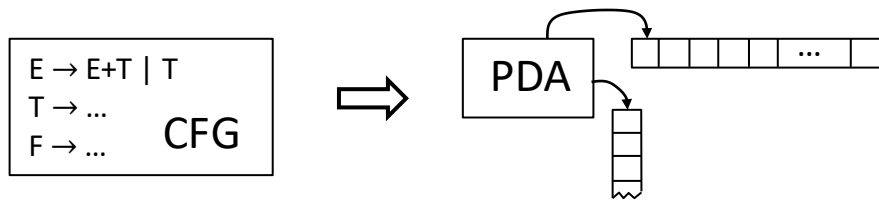
IDEA:

- The PDA will accept the input string if G_2 generates that input (if it can derive it)
- PDA begins with starting variable on the stack
- **Problem:** which derivation to do?
- Nondeterminism allows it to guess the sequence of correct substitutions.
- Substitution after substitution it keeps the intermediate generated strings on stack.
- When it reaches a string of terminals it accepts by comparing it with the input and it's the same.

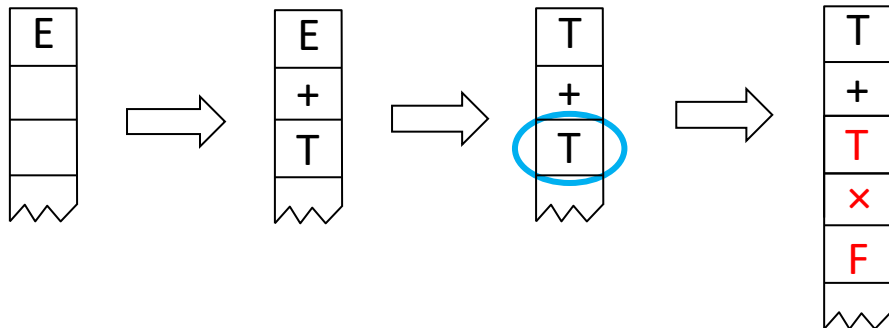
Converting CFGs to PDAs (2)

Theorem: If A is a CFL then some PDA recognizes A

Proof: Convert A 's CFG to a PDA



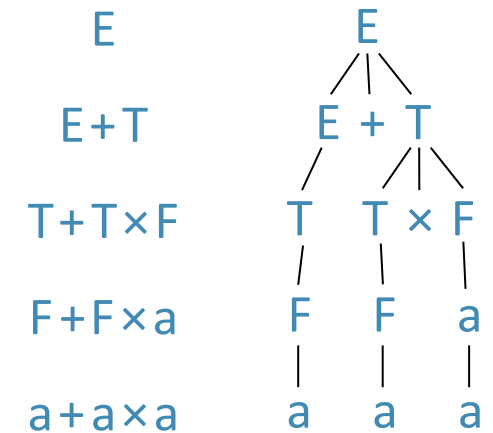
IDEA: PDA begins with starting variable and guesses substitutions. It keeps intermediate generated strings on stack. When done, compare with input.



Input:

a	+	a	x	a
---	---	---	---	---

G_2 $E \rightarrow E+T \mid T$
 $T \rightarrow T \times F \mid F$
 $F \rightarrow (E) \mid a$

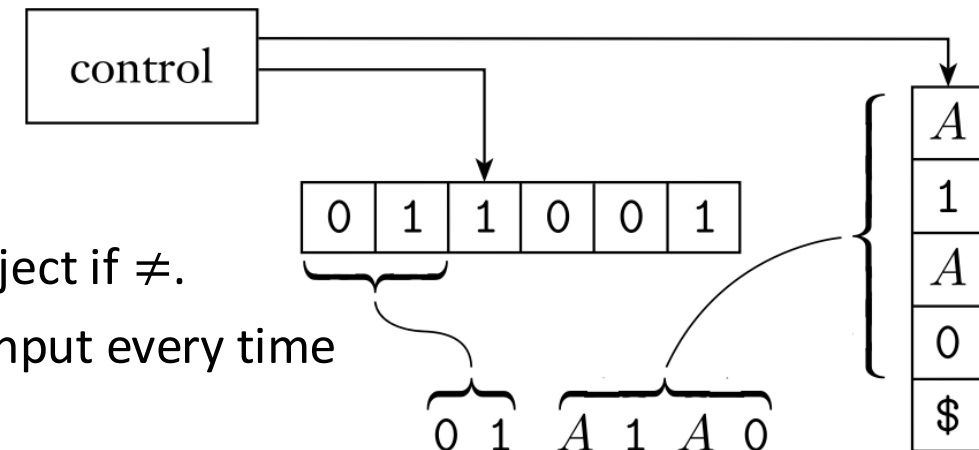


Problem! The PDA can only access the top of stack!

Instead, only substitute variables when on the top of stack.

If a terminal is on the top of stack, pop it and compare with input. Reject if $\neq$.

(Only keep **part** of the intermediate string on the stack and read the input every time a terminal symbol is on the top of the stack



Converting CFGs to PDAs (contd)

Theorem: If A is a CFL then some PDA recognizes A

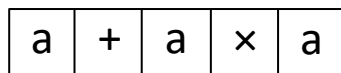
Proof construction: Convert the CFG for A to the following PDA.

- 1) Push the \$ and the start symbol on the stack.
- 2) If the top of stack is

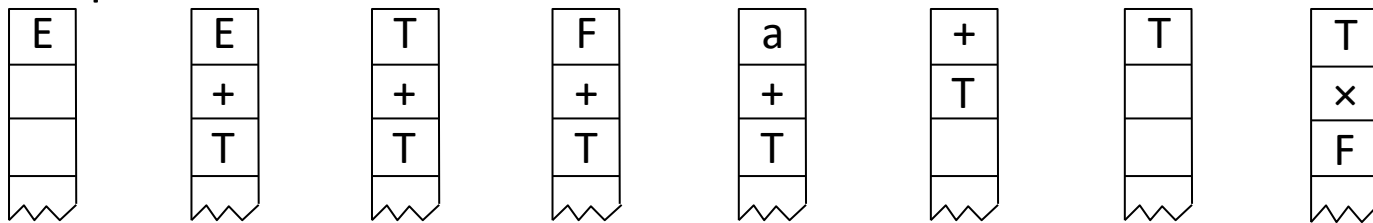
Variable: replace with right hand side of rule (nondeterministic choice).

Terminal: pop it and match with next input symbol (if they match repeat, else *reject*).

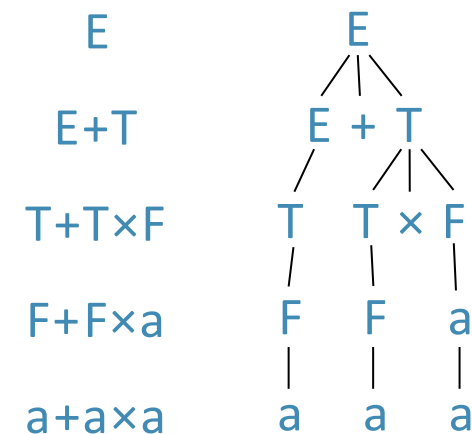
- 3) If the stack is empty (\$), *accept*.



Example:



?

$$G_2 \quad \begin{array}{l} E \rightarrow E+T \mid T \\ T \rightarrow T \times F \mid F \\ F \rightarrow (E) \mid a \end{array}$$


Resulting PDA (conversion CGA -> PDA)

Construct a PDA $P(Q, \Sigma, \Gamma, \delta, q_{start}, F)$ from CFG:

$$Q = \{q_{start}, q_{loop}, q_{accept}\} \cup E$$

δ :

$$1. \delta(q_{start}, \epsilon, \epsilon) = (q_{loop}, S\$)$$

2. loop:

a) If pop is variable:

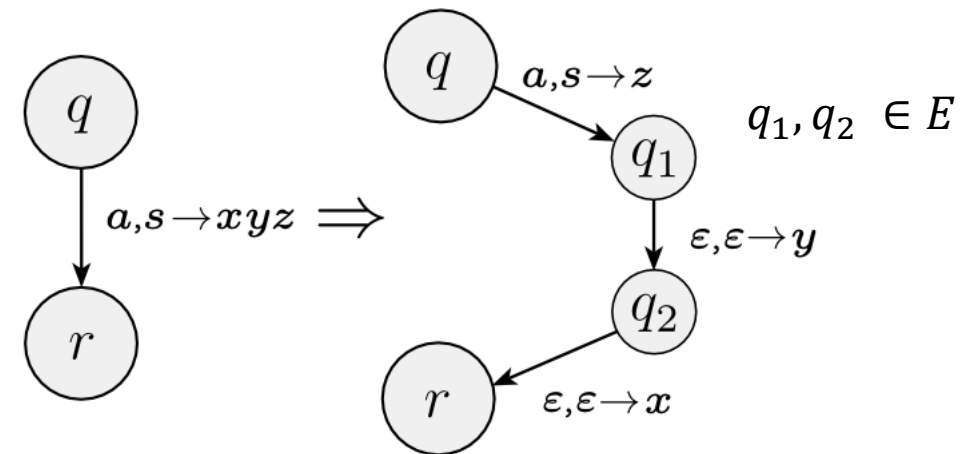
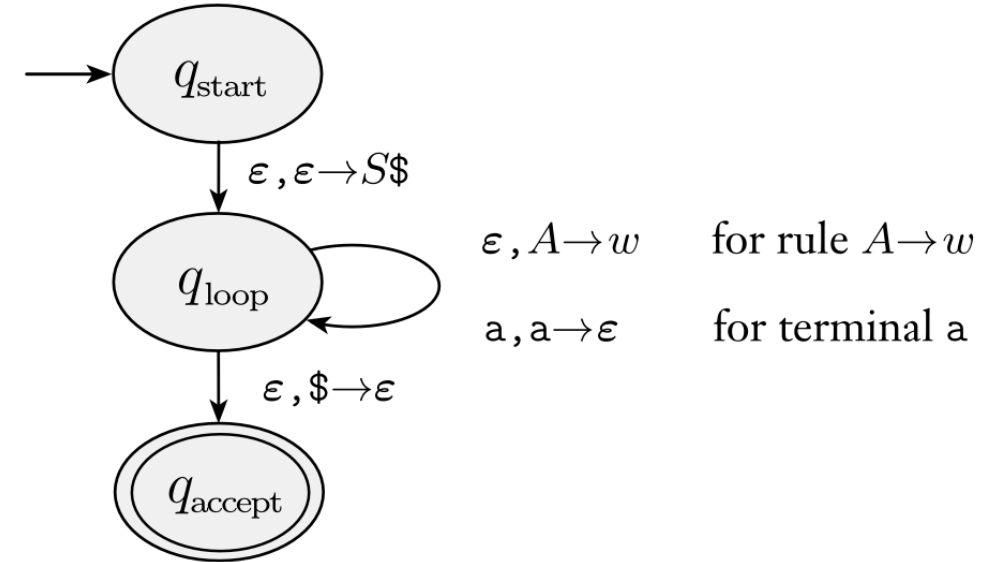
$$\bullet \delta(q_{loop}, \epsilon, A) = \{(q_{loop}, w) \mid \text{where } A \rightarrow w \text{ is a rule in } R\}$$

b) If pop is terminal:

$$\bullet \delta(q_{loop}, a, a) = (q_{loop}, \epsilon)$$

3. If pop is \$

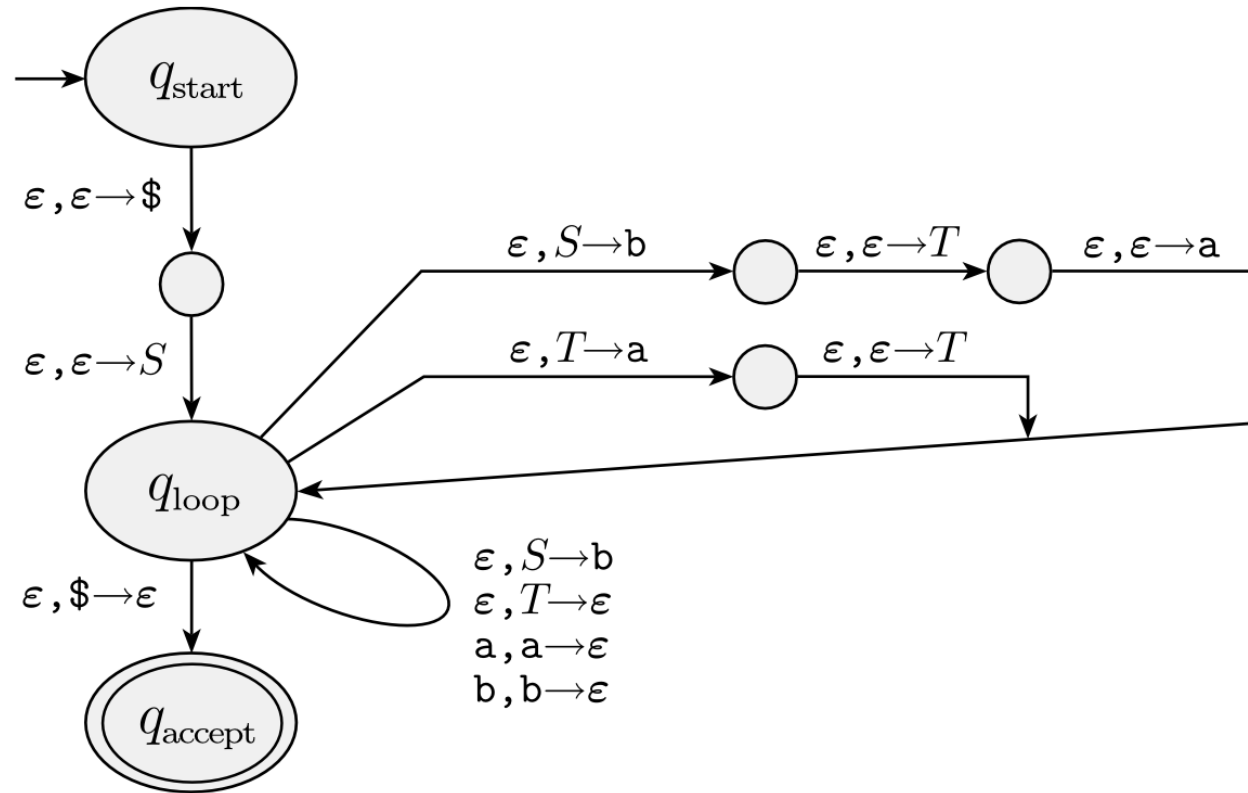
$$\bullet \delta(q_{loop}, \epsilon, \$) = (q_{accept}, \epsilon)$$



Converting CFGs to PDAs: Example

Construct a PDA P from the following language G :

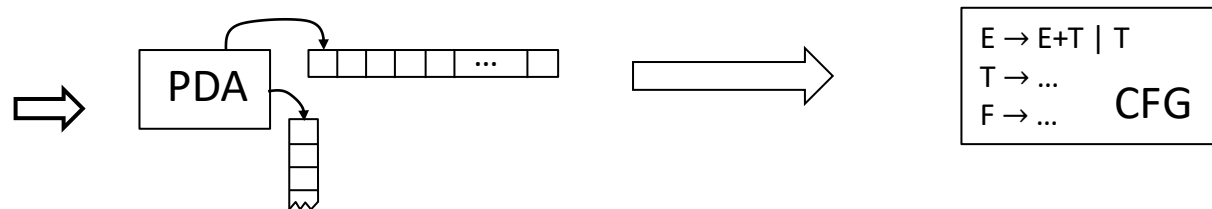
$$\begin{aligned} S &\rightarrow aTb \mid b \\ T &\rightarrow Ta \mid \varepsilon \end{aligned}$$



Converting PDAs to CFGs (1)

Theorem: If a PDA recognizes A , then it is context free.

Proof: Convert A 's PDA P to a CFG G



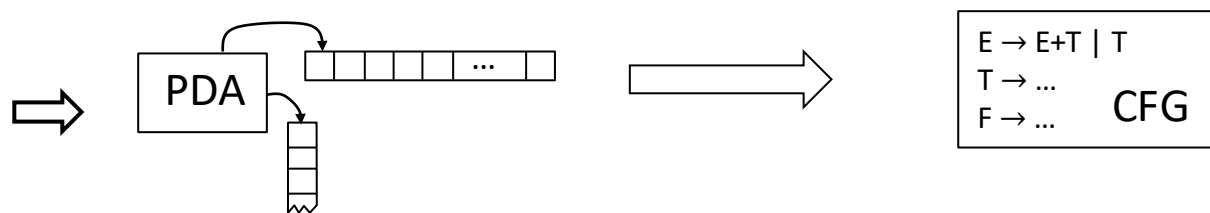
PROOF IDEA:

- G will generate all the strings that P accepts (if it goes from start state to accept state on that input)
- We simplify P to make things simpler for us:
 1. P will have a single accept state
 2. P empties its stack before accepting
 3. Each transition in P either *pushes* a symbol on the stack **or** *pops* one (not both, not neither)
 - Features 1 and 2 are easy. For feature 3 we use a new state either to:
 - push first and pop after or
 - if neither happens to simulate a push of a symbol and then pop it again
 4. This means that the **first** operation on the stack of P is a push and the **last** a pop (cond. 2)

Converting PDAs to CFGs (2)

Theorem: If a PDA recognizes A , then it is context free.

Proof: Convert A 's PDA P to a CFG G



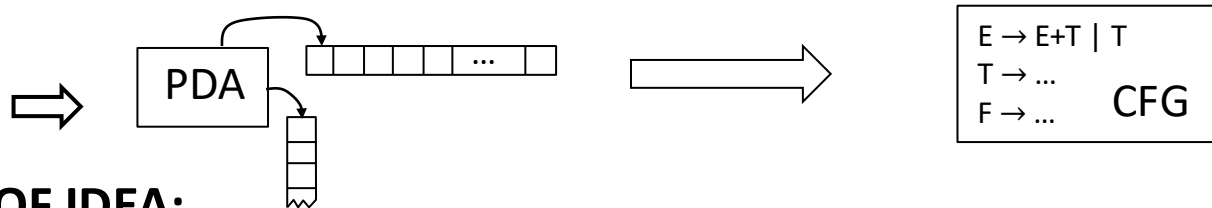
PROOF IDEA:

- G will generate all the strings that P accepts (if it goes from start state to accept state on that input)
- For each pair of states p and q , G will have a variable $A_{p,q}$
- $A_{p,q}$ will generate **all** strings x that take P from p (with an **empty stack**) to q (with an **empty stack**).
- During the computation of x there are two possibilities (stack never emptied (1) or emptied (2)):
 1. If the last symbol popped during computation of x is the same as first pushed:
=> insert rule $A_{p,q} \rightarrow aA_{r,s}b$ (a is the first symbol read, and b the last read)
 2. If the last symbol popped when reading x is different than the first pushed:
=> insert rule $A_{p,q} \rightarrow A_{p,r}A_{r,q}$ (r is the state where the stack becomes empty again)

Converting PDAs to CFGs (3)

Theorem: If a PDA recognizes A , then it is context free.

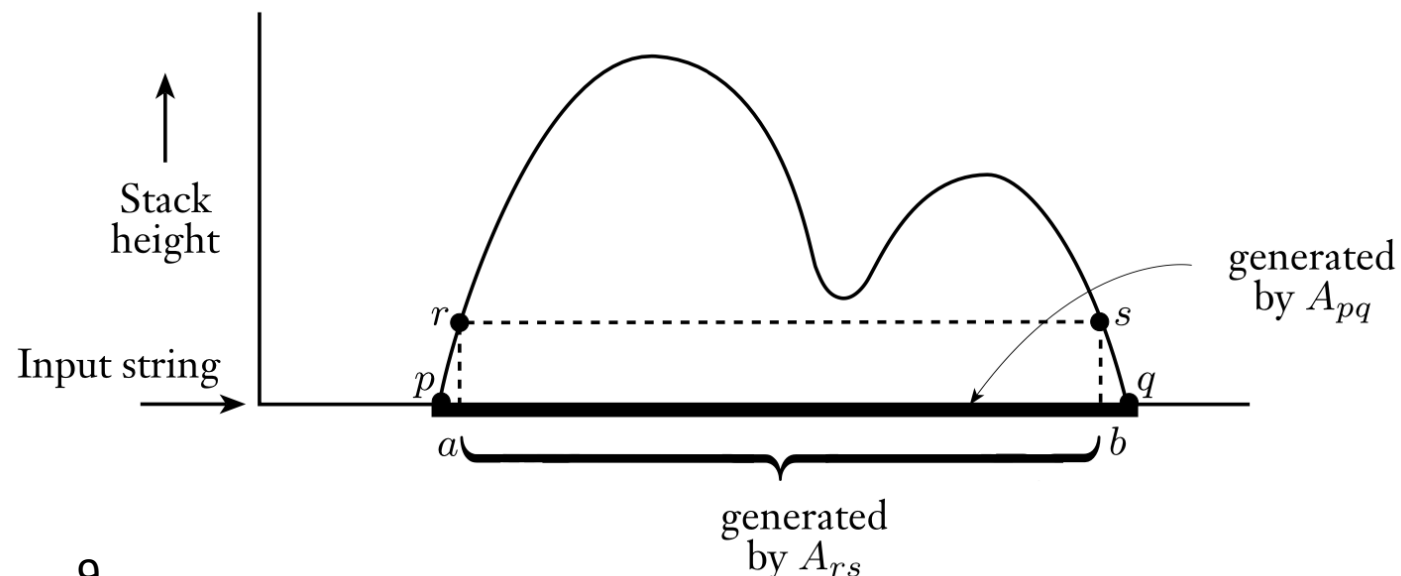
Proof: Convert A 's PDA P to a CFG G



PROOF IDEA:

- During the computation of x there are two possibilities:
 - If the last symbol popped during computation of x is the same as first pushed:
 $\Rightarrow$ insert rule $A_{p,q} \rightarrow aA_{r,s}b$

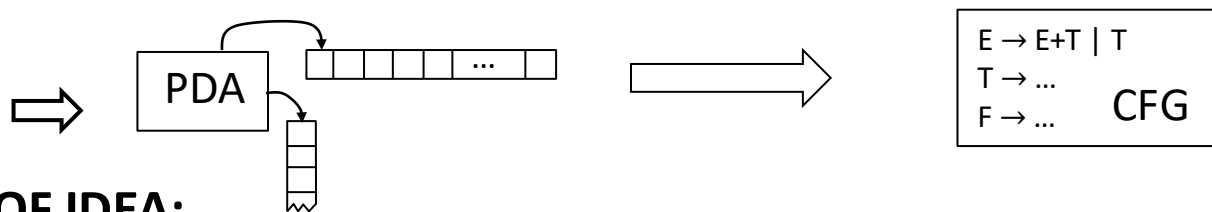
- a is the first symbol read
- b the last symbol read
- r is the state reached from p by reading a
- s is the state preceding q which is reached from s by reading b



Converting PDAs to CFGs (4)

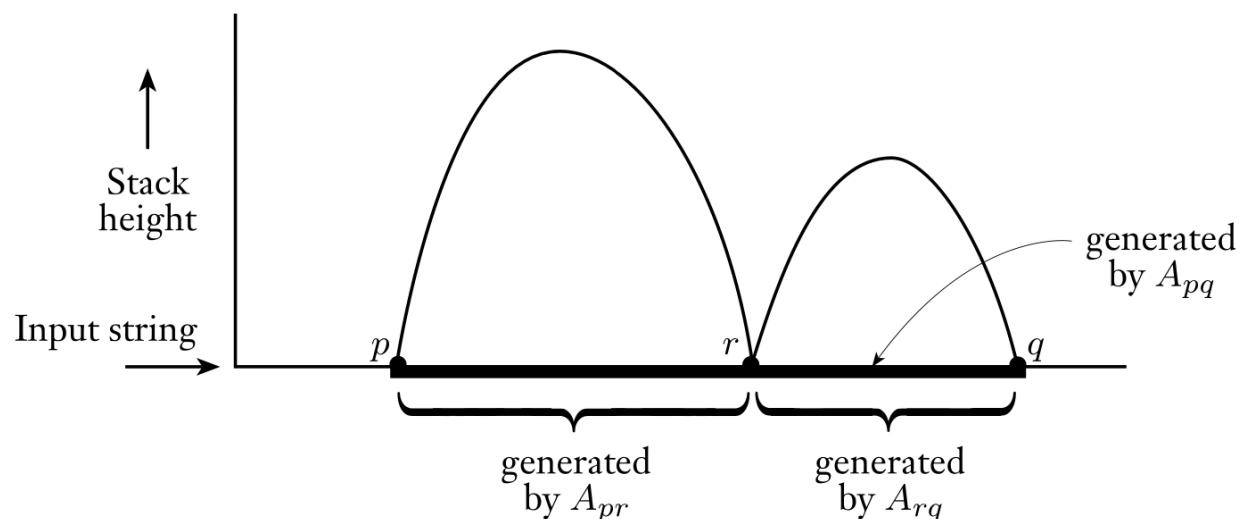
Theorem: If a PDA recognizes A , then it is context free.

Proof: Convert A 's PDA P to a CFG G



PROOF IDEA:

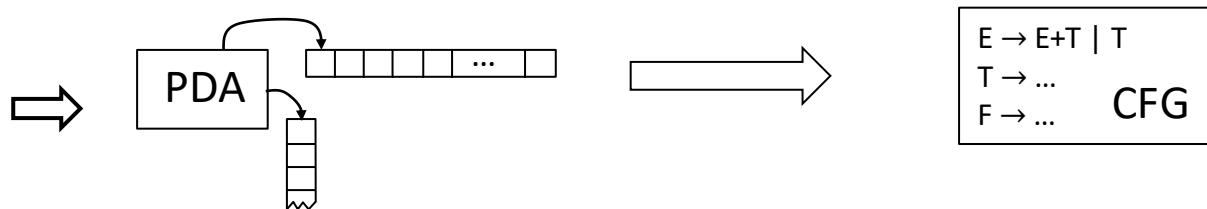
- During the computation of x there are two possibilities:
 - If the last symbol popped when reading x is different than the first pushed:
(stack becomes empty at some point)
 $\Rightarrow$ insert rule $A_{p,q} \rightarrow A_{p,r}A_{r,q}$ (r is the state where the stack becomes empty again)



Converting PDAs to CFGs (5)

Theorem: If a PDA recognizes A , then it is context free.

Proof: Convert A 's PDA P to a CFG G



Construction:

- Let $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{\text{accept}}\})$ and construct G . The variables are $\{A_{pq} \mid p, q \in Q\}$. The start variable is $A_{q_0, q_{\text{accept}}}$. The rules are as follows:

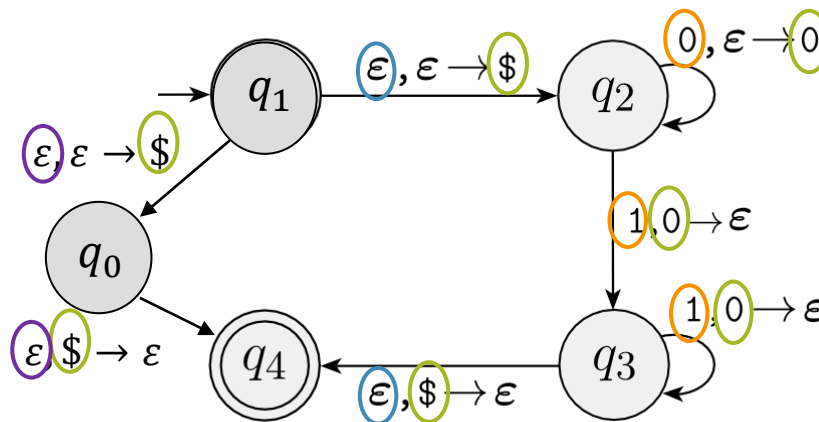
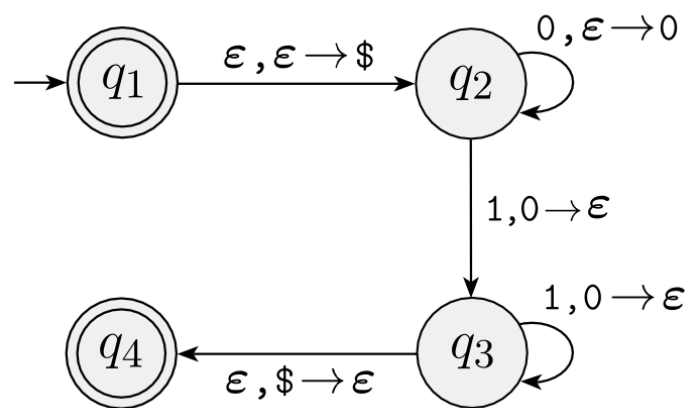
1. For each $p, q, r, s \in Q$, $u \in \Gamma$, and $a, b \in \Sigma_\epsilon$, if $\delta(p, a, \epsilon)$ contains (r, u) and $\delta(s, b, u)$ contains (q, ϵ) , put the rule $A_{pq} \rightarrow aA_{rs}b$ in G .
2. For each $p, q, r \in Q$, put the rule $A_{pq} \rightarrow A_{pr}A_{rq}$ in G .
3. Finally, for each $p \in Q$, put the rule $A_{pp} \rightarrow \epsilon$ in G .

Converting PDAs to CFGs: Example

Construction:

- Let $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{\text{accept}}\})$ and construct G . The variables are $\{A_{pq} \mid p, q \in Q\}$. The start variable is $A_{q_0, q_{\text{accept}}}$. The rules are as follows:
 - For each $p, q, r, s \in Q$, $u \in \Gamma$, and $a, b \in \Sigma_\epsilon$, if $\delta(p, \textcircled{a}, \epsilon)$ contains $(r, \textcircled{u})$ and $\delta(s, \textcircled{b}, \textcircled{u})$ contains (q, ϵ) , put the rule $A_{pq} \rightarrow aA_{rs}b$ in G .
 - For each $p, q, r \in Q$, put the rule $A_{pq} \rightarrow A_{pr}A_{rq}$ in G .
 - Finally, for each $p \in Q$, put the rule $A_{pp} \rightarrow \epsilon$ in G .

Example: P for the language $\{0^n 1^n \mid n \geq 0\}$



G :

q_1 is start state (not q_0 !!)

$$A_{q_1 q_4} \rightarrow \epsilon A_{q_0 q_0} \epsilon \mid \epsilon A_{q_2 q_3} \epsilon$$

$$A_{q_2 q_3} \rightarrow 0 A_{q_2 q_2} 1 \mid 0 A_{q_2 q_3} 1$$

...

(rules 2)

...

$$A_{q_0 q_0} \rightarrow \epsilon$$

$$A_{q_2 q_2} \rightarrow \epsilon$$

...

Remember: In the general case, you may need to add a new state before the accept state to make sure the stack is empty!

Converting PDAs to CFGs: Proof

Construction rules:

1. For each $p, q, r, s \in Q$, $u \in \Gamma$, and $a, b \in \Sigma_\varepsilon$, if $\delta(p, a, \varepsilon)$ contains (r, u) and $\delta(s, b, u)$ contains (q, ε) , put the rule $A_{pq} \rightarrow aA_{rs}b$ in G .
2. For each $p, q, r \in Q$, put the rule $A_{pq} \rightarrow A_{pr}A_{rq}$ in G .
3. Finally, for each $p \in Q$, put the rule $A_{pp} \rightarrow \varepsilon$ in G .

Theorem ($\Rightarrow$) : If A_{pq} generates x , then x can bring P from p (with empty stack) to q (with empty stack)

Proof by induction (on length of derivation)

Base case (derivation has 1 step): the rule must have no variables on the right (1 step)

$\Rightarrow$ only possible rule in G is $A_{pp} \rightarrow \varepsilon$, which P can do easily (go from p to p on input ε)

Inductive case (assume true of steps $\leq k$): prove true $A_{pq} \Rightarrow^* x$ in $k + 1$ steps. Then the first step is:

1. $A_{pq} \rightarrow aA_{rs}b$: then $x = ayb$, and P can derive $A_{rs} \Rightarrow^k y$ by inductive hyp. starting and ending with empty stack. It can $p \rightarrow (r, u)$ reading a , then get through y with u at the top of the stack; then $(s, b, u) \rightarrow (q, \varepsilon)$ by reading b and popping u off the stack (i.e., empty stack to empty stack).
2. $A_{pq} \rightarrow A_{pr}A_{rq}$: then $x = yz$, and since $A_{pr} \Rightarrow^{\leq k} y$ and $A_{rq} \Rightarrow^{\leq k} z$ are both $\leq k$ steps by inductive hyp. are done by P with empty stacks at beginning and end. Nothing else to do.

Converting PDAs to CFGs: Proof

Construction rules:

1. For each $p, q, r, s \in Q$, $u \in \Gamma$, and $a, b \in \Sigma_\epsilon$, if $\delta(p, a, \epsilon)$ contains (r, u) and $\delta(s, b, u)$ contains (q, ϵ) , put the rule $A_{pq} \rightarrow aA_{rs}b$ in G .
2. For each $p, q, r \in Q$, put the rule $A_{pq} \rightarrow A_{pr}A_{rq}$ in G .
3. Finally, for each $p \in Q$, put the rule $A_{pp} \rightarrow \epsilon$ in G .

Theorem ($\Leftarrow$) : If x can bring P from p (with empty stack) to q (with empty stack), then A_{pq} generates x .

Proof by induction (on the number of computation steps of P)

Base case (0 steps): P goes nowhere so stays in current state p

$\Rightarrow G$ by construction has rule $A_{pp} \rightarrow \epsilon$

Inductive case (assume true for computation lengths $\leq k$): prove true for length $k + 1$ steps. Then:

1. **First symbol pushed is the same as last popped**: let this symbol be u . Let a first symbol read, b the last one, r the state after first move and s the state before the last move. Then $\delta(p, a, \epsilon) = (r, u)$ and $\delta(s, b, u) = (q, \epsilon)$. So the rule $A_{pq} \rightarrow aA_{rs}b$ is in G . Let $x = ayb$. Since we've done 2 computation steps, the remaining ones are $k - 1$, which G can generate by ind. hypothesis. Hence, $A_{r,s} \Rightarrow^* y$ and $A_{p,q} \Rightarrow^* x$
2. If the **stack becomes empty before the end of the computation**, let r be that state. Then the computations $p \rightarrow r$ and $r \rightarrow q$ contain at most k steps and $A_{pq} \rightarrow A_{pr}A_{rq}$ so by ind. hyp the string is generated by G .

Equivalence of CFGs and PDAs

Theorem: A is a CFL iff* some PDA recognizes A

↔ Done.

Done

Check-in 4.3

Is every Regular Language also a Context Free Language?

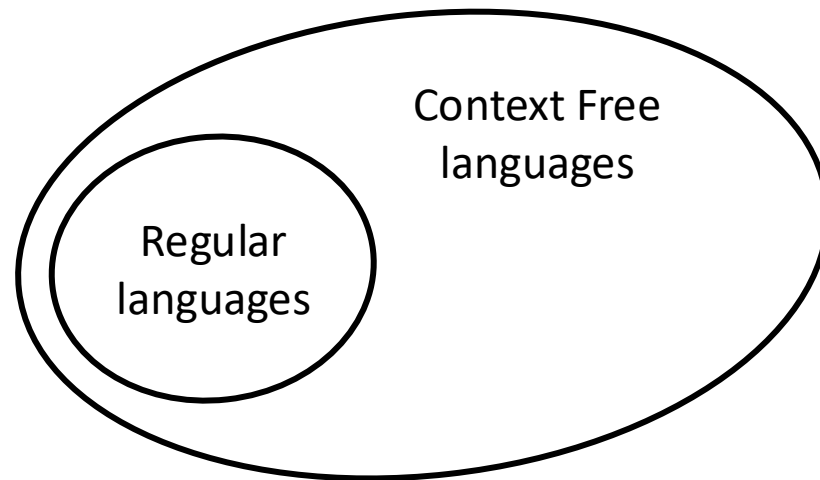
- (a) Yes
- (b) No
- (c) Not sure

Check-in 4.3

Recap

Corollary: Every regular language is context free.

	Recognizer	Generator
Regular language	DFA or NFA	Regular expression
Context Free language	PDA	Context Free Grammar



Quick review of today

1. Gave conversion of CFGs to PDAs and viceversa